

Reflection Probes

Reflection probes, which provide both bounce lighting and reflections, consist of two major parts:

- The actual reflection cube map.
- The occlusion and diffuse compensation grid.

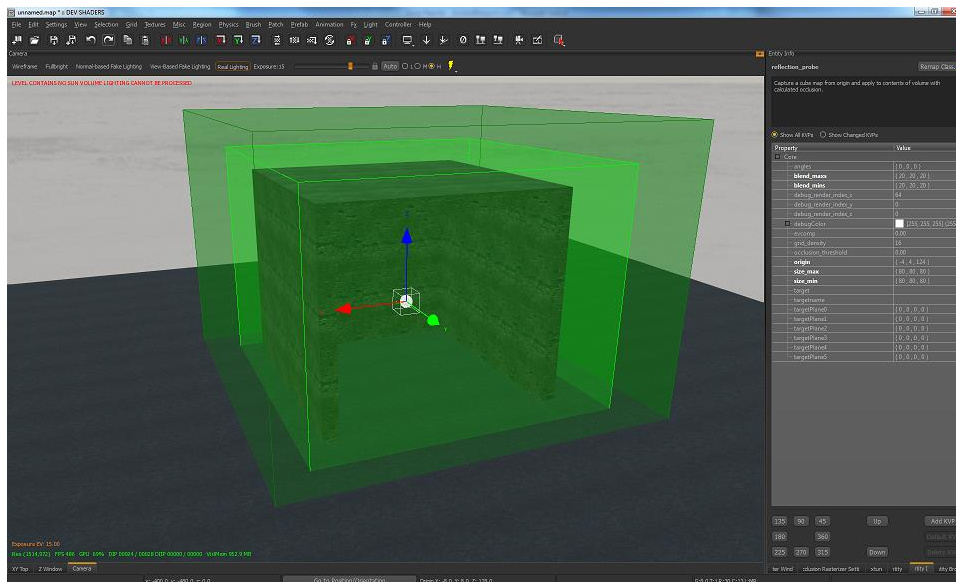
Probe Placement and Sizing

The probe size is adjusted by the size_max (x,y,z) and size_min (-x,-y,-z). The middle mouse wheel will increase by increments of 10. The size can also be scaled using the KVP handles that appear when a single probe is selected, or by dragging in the 2D window similar to regular brushes.

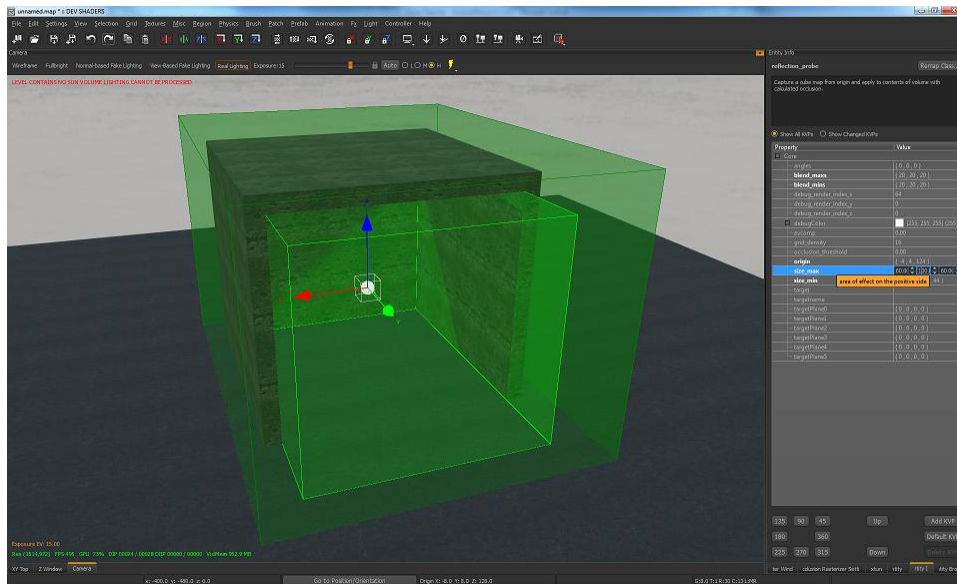
BUT, the easier way is to use snap to geo function. With the probe selected ALT+Left Click will snap the probe bounds to the surface you've clicked on. Turning on "Show Unselected Probe Boxes" in the View menu will allow you to snap directly to the surfaces of other probes as well. It is advisable to map this function to a key and toggle it on only as needed.

Hold ALT and left click all the surfaces surrounding you! ***(ALT+Right Click for Reflection Planes)***

[See probe_snap.gif in your doc_modtools\probe_gifs folder for video example]



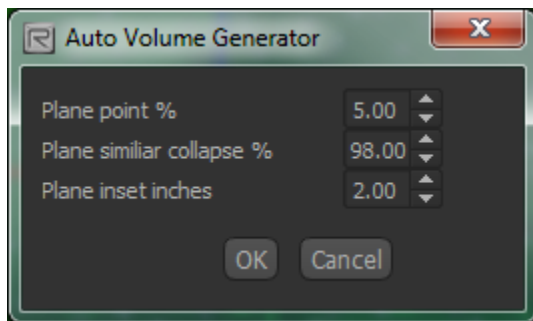
Probes can be asymmetrical, and the probe location doesn't even need to be within the box. This can be useful if you need to render the cube map from a location that isn't directly related to the box bounds. The probe location can be moved independently of the box by holding Ctrl+Shift and either dragging a move handle in the 3D window, or clicking and dragging within the probe box on the 2D window.



[See probe_nonBox.gif in your doc_modtools\probe_gifs folder for video example]

Reflection Planes

Reflection planes tell the probe where the walls are, so reflections will parallax correctly. There are only six planes, so it is necessary to choose the six most important walls around the probe. The "W" key will attempt to choose all six automatically, bringing up the settings dialog pictured below.



Reflection planes can also be set by using the snap to geo function.

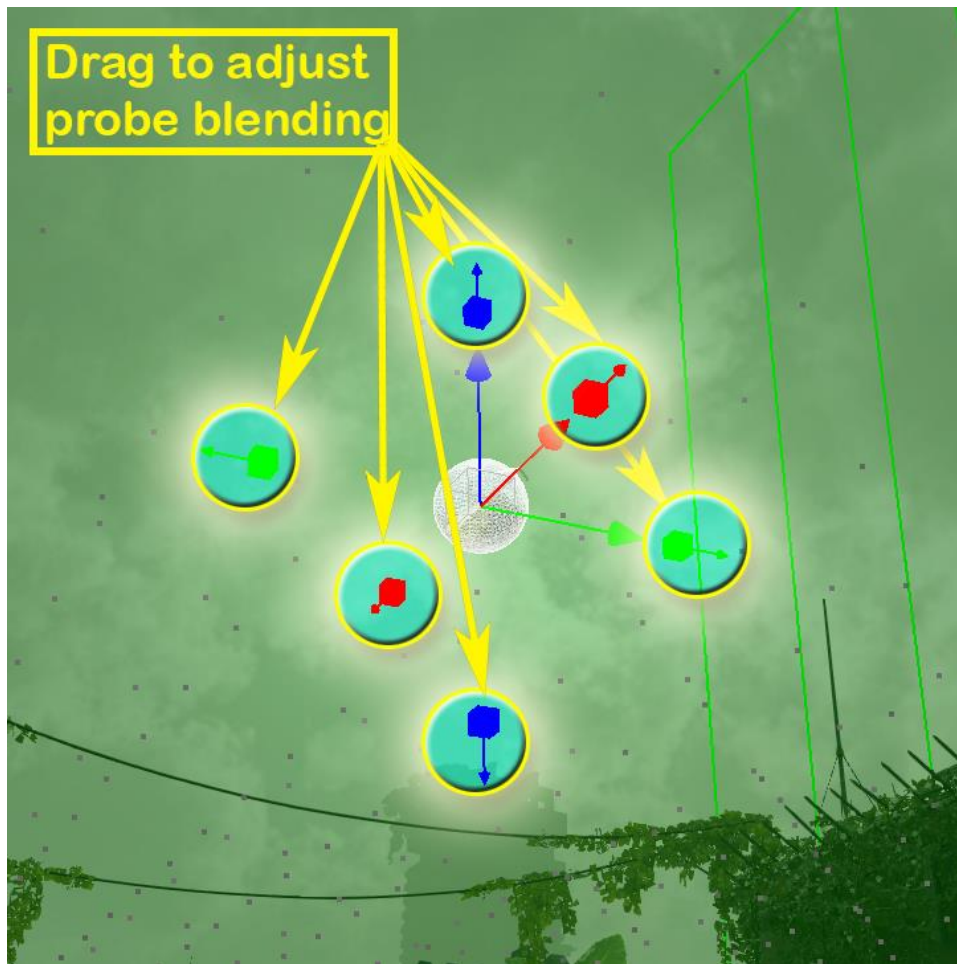
With the Probe selected, hold ALT+Right click on the surfaces you want a reflection plane to snap to.

[See probe_snap_planes.gif in your doc_modtools\probe_gifs folder for video example]

Planes will snap with an inset of 2 units to place them right in front of walls.

The screenshot displays the Unreal Engine 4.26.2 interface. The main viewport shows a 3D scene with a green cube and a wooden platform. The scene is overlaid with a grid of red, green, and blue lines. The right-hand side shows the 'Reflections' panel with various settings and a 'Purple plane' highlighted in the 'Targets' list.

Probes can be blended by adjusting the `blend_maxs` and `blend_mins`. Clicking and moving the middle mouse wheel will increase by increments of 10.



Probes fade from fully blended in to fully blended out between the inner and outer box. Also, probes that have a higher grid_density, are smaller, or are less occluded will blend over other overlapping probes.

[See probeBlends.gif in your doc_modtools\probe_gifs folder for video example]

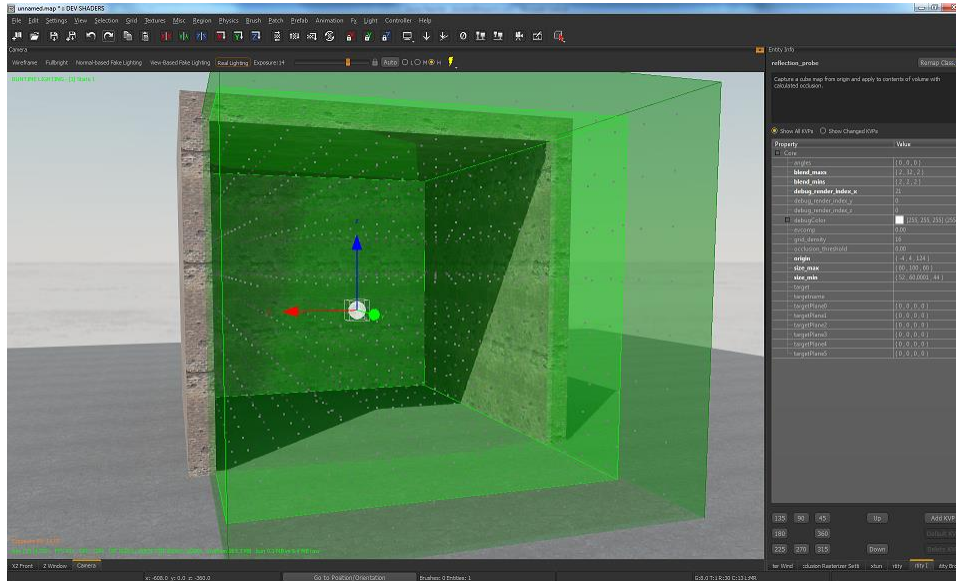
Probe Rotation

Probes can be rotated to fit a space that is off grid, by using the rotation tool as with other brushes. You can also match the rotation of any brush you mouse over. Use the key combo SHIFT+ALT+R with the mouse over the plane you wish to match to.

[See probe_snap_angles.gif in your doc_modtools\probe_gifs folder for video example]

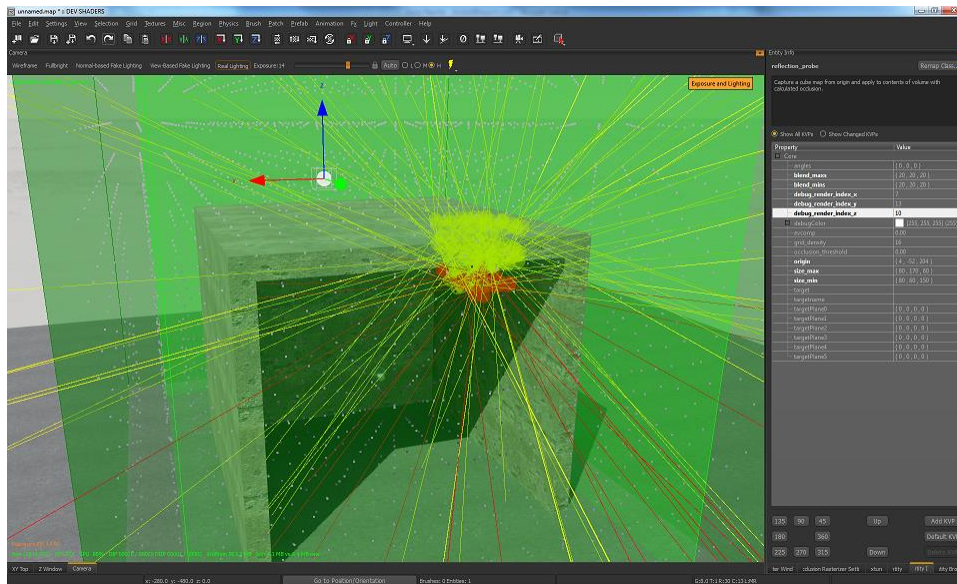
GI Calculation

GI bounce is calculated for every point within a regular grid inside the probe bounds. The "grid_density" KVP specifies the maximum resolution of this grid along the longest sides, with shorter sides rounding to the closest power of two in resolution. The grids are shown as small squares once calculated during lighting calculation.



For each point in the grid, Radiant casts a number of random rays. If these rays are at all visible to any nearby probe, then color data is read from them and used to calculate bounce lighting. Therefore it is important that all areas where accurate lighting are required are visible to the center point of at least one nearby probe. Also, only the 8 closest probes are used to collect this color data, so if too many probes are provided, Radiant may give up looking before finding a probe with decent visibility. Probes can be flagged as "virtual" and will appear with a red surface. These probes will contribute visibility data to other probes during baking, but do not have an accompanying volume and do not apply reflections to the scene themselves. For this reason they do not take up a probe slot in game, either.

You can debug the occlusion rays using the "debug_render_index" KVPs. If the index is set to any valid coordinate when lighting is built, all of the rays for the chosen probe will be stored for debugging. The easy way to insure this will happen is set all three KVPs to zero. Once lighting is built you can use the mouse wheel to scroll through the grid points and see the test rays. Red rays are occluded while yellow rays are visible. Once done it is a good idea to set "debug_render_index_x" back to zero because if debug rays are stored for all the probes in a level Radiant might run out of memory.



The Global Probe

The global probe provides ambient lighting based on how visible the sky is to surfaces. It uses a volume texture just like the local probes. The size and resolution of the grid is controlled by the size of the `sun_volume` and the `grid_density` parameter on it. An overall minimum occlusion parameter is also present. When set non-zero it will prevent unlit and fully occluded areas from going completely dark, and when set to 1 it essentially turns the global probe to an un-occluded view of the sky. Lighting has to be rebuilt for the setting to take effect.

The global probe only blends in outside of the attenuation range of all other probes.

In the images below the level is shown first with the global probe disabled, and then with it enabled. The main benefit is shading between the buildings and near where they meet the ground, as well as subtle directional shading even on the unlit side.

